**LAPORAN PRAKTIKUM**

**PEMROGRAMAN DASAR**

**MODUL 2**

Dosen Pengampu: Dede Irawan, M.Kom


**Disusun oleh :**

| | | |
|---|---|---|
| **Nama** | **:** | **Eti Sunanti** |
| **NIM** | **:** | **20250910083** |
| **Kelas** | **:** | **SINFC-02-2025** |



# SISTEM INFORMASI

# FAKULTAS ILMU KOMPUTER

# UNIVERSITAS KUNINGAN

**2026**

## KATA PENGANTAR

Puji dan syukur saya panjatkan kehadirat Tuhan Yang Maha Esa atas segala rahmat-Nya sehingga laporan praktikum Desain Web Modul 2 ini dapat diselesaikan dengan baik. Laporan ini disusun sebagai bukti pelaksanaan praktikum di Program Studi Sistem Informasi, Universitas Kuningan, yang berfokus pada materi *version control* menggunakan Git dan GitHub. Melalui modul ini, saya belajar mengelola riwayat perubahan kode serta menyimpan proyek secara online. Semoga laporan ini dapat bermanfaat bagi pembaca, dan kritik maupun saran sangat saya harapkan demi penyempurnaan ke depannya.

Kuningan, 29 Oktober 2026

# DAFTAR ISI

**KATA PENGANTAR**..... ii
**DAFTAR ISI**..... iii
**BAB I**..... 4
**PRETEST** ..... **4**
**BAB II** ..... 5
**PEMBAHASAN** ..... **5**
**BAB III**..... 6
**POSTEST** ..... **6**
**BAB IV**..... 7
**TUGAS**..... **7**
**BAB V** ..... 10
**KESIMPULAN** ..... **10**
**BAB V** .....Error! Bookmark not defined.
**KESIMPULAN** .....Error! Bookmark not defined.

# BAB I

# PRETEST

1. Apa tujuan dari menggunakan version control dalam pengembangan perangkat lunak?
   - Tujuan utamanya adalah untuk mencatat dan melacak setiap perubahan pada file/kode program dari waktu ke waktu. Hal ini memudahkan pengembang untuk kembali ke versi sebelumnya jika terjadi kesalahan (*rollback*), mempermudah kolaborasi antar tim tanpa saling menimpa kode, serta memungkinkan eksperimen fitur baru secara aman melalui cabang (*branch*).

2. Jelaskan perbedaan antara git add dan git commit!
   - git add: Berfungsi untuk memindahkan perubahan file dari *working directory* ke *staging area* (menandai file yang siap untuk disimpan).
   - git commit: Berfungsi untuk menyimpan perubahan dari *staging area* secara permanen ke *repository* lokal beserta pesan catatan log perubahan.

3. Bagaimana cara membuat branch baru di Git?
   - git branch <nama_branch_baru> (untuk membuat cabang baru).
   - git checkout -b <nama_branch_baru> (untuk membuat cabang baru sekaligus berpindah ke dalamnya).

# BAB II

# PEMBAHASAN

PRAKTIKUM

1. Membuat Akun GitHub: Mendaftar di *github.com* dan melakukan verifikasi email.
   - Akun GitHub digunakan sebagai *remote repository* di cloud.

2. Instalasi & Konfigurasi Git: Mengatur nama dan email lokal:
   git config --global user.name "Nama Pengguna"
   git config --global user.email "email@domain.com"
   - Mendaftarkan identitas pembuat ‘commit’ pada database Git.

3. Inisialisasi & Commit Lokal:
   git init
   git add .
   git commit -m "Initial commit"
   - `git init` membuat repository lokal, `git add` menyiapkan file, dan `git commit` menyimpan riwayat perubahan.

# BAB III

# POSTEST

1. Apa fungsi dari git merge dalam workflow Git?
   - git merge berfungsi untuk menggabungkan riwayat perubahan (*commit history*) dari satu cabang (*branch*) ke cabang lainnya (misalnya menggabungkan *branch* pengerjaan fitur baru ke *branch* utama/main).

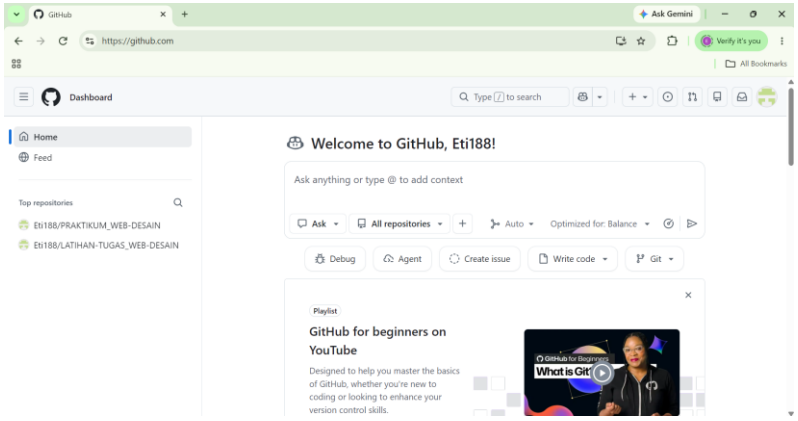
2. Jelaskan cara mengatasi konflik saat melakukan merge!
   a. Buka file yang mengalami konflik (ditandai oleh Git dengan tanda <<<<<<<, =======, dan >>>>>>>).
   b. Pilih kode mana yang ingin dipertahankan (*Accept Current Change*, *Accept Incoming Change*, atau gabungan keduanya).
   c. Hapus tanda pembatas konflik dari Git.
   d. Simpan file, lalu jalankan git add <nama_file> untuk menandai bahwa konflik telah diselesaikan.
   e. Lakukan *commit* penyelesaian konflik dengan git commit -m "Fix merge conflict".

3. Tulis langkah-langkah untuk menambahkan file baru ke repository Git dan melakukan commit.
   a. Membuat atau mengubah file di folder proyek (misal Latihan_02.html).
   b. Periksa status file dengan perintah git status.
   c. Masukkan file ke *staging area* menggunakan perintah git add Latihan_02.html (atau git add . untuk semua file).
   d. Simpan perubahan ke *repository* lokal dengan perintah git commit -m "Menambahkan file Latihan_02.html".

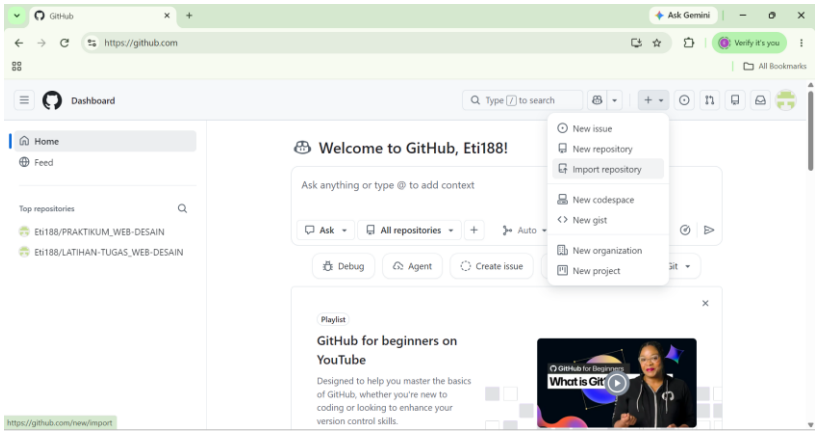
# BAB IV

# TUGAS

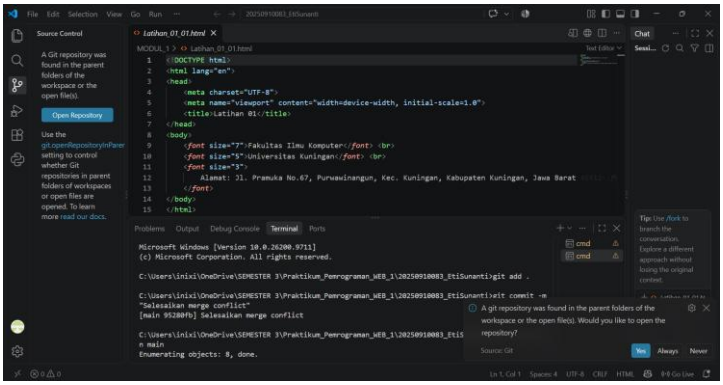
1. Buatlah repository Git untuk proyek web dan dokumentasikan proses pembuatan, commit, dan push ke GitHub.
   a. Pembuatan Repository di GitHub:
      1. Akses halaman web (https://github.com/) dan masuk (*sign in*) menggunakan akun yang telah dibuat.




      2. Klik ikon "+" di pojok kanan atas, lalu pilih New repository.




      3. Mengisi *Repository name* dengan nama pweb1, memilih visibilitas *Public*, lalu klik tombol Create repository.
   b. Inisialisasi dan Commit di Lingkungan Lokal:



1. Membuka proyek di Visual Studio Code, lalu membuka terminal
2. Menginisialisasi folder proyek menjadi *repository* Git lokal:
   git init
3. Menambahkan seluruh berkas praktikum (Modul 1) ke *staging area*:
   git add .
4. Menyimpan perubahan ke *repository* lokal beserta catatan *commit*:
   git commit -m "Initial commit: Menambahkan berkas praktikum Modul 1"

c. Menghubungkan dan Push ke GitHub:
   1. Menghubungkan *repository* lokal ke *remote repository* GitHub:
      git remote add origin https://github.com/username/pweb1.git
   2. Mengubah nama cabang (*branch*) utama menjadi main:
      git branch -M main
   3. Mengirimkan (*push*) seluruh riwayat *commit* ke GitHub:
      git push -u origin main

2. Rancanglah workflow Git yang melibatkan branch untuk fitur baru dan integrasi dengan branch utama.

   Dalam pengembangan perangkat lunak, penggunaan *branch* (cabang) bertujuan untuk memisahkan proses pengerjaan fitur baru agar tidak mengganggu kode utama yang sudah stabil.

   - Struktur Cabang (*Branch*):
     - main: Cabang utama yang menyimpan versi kode program paling stabil (siap rilis/produksi)
     - feature-modul2: Cabang khusus yang dibuat untuk mengerjakan tugas atau penambahan fitur baru.
   - Alur Kerja (*Workflow*) Pengembangan Fitur:
     1. Membuat dan berpindah ke cabang fitur baru:
        git checkout -b feature-modul2
     2. Melakukan perubahan atau penambahan berkas program pada cabang tersebut.
     3. Menyimpan perubahan ke *staging area* dan melakukan *commit*:
        git add .
        git commit -m "Menambahkan pengerjaan Modul 2"
     4. Berpindah kembali ke cabang utama (main):
        git checkout main
     5. Menggabungkan (*merge*) perubahan dari cabang fitur ke cabang utama:
        git merge feature-modul2
     6. Mengunggah hasil penggabungan terbaru ke GitHub:
        git push origin main

3. Simpan seluruh praktikum pada modul sebelumnya kedalam repository Git yang sudah anda buat.
    - Setelah semua file di add, di commit, lalu dipush menggunakan perintah git ke reposting di GitHub. Semua file sudah tersimpan di akun GitHub

# BAB V

# KESIMPULAN

Melalui praktikum Modul 2 ini, dapat disimpulkan bahwa Git merupakan sistem kontrol versi yang sangat penting untuk melacak dan mengelola riwayat perubahan kode program secara rapi. Penggunaan perintah dasar seperti git init, git add, git commit, dan git push memungkinkan penyimpanan berkas dari lingkungan lokal ke *repository* remote secara terstruktur. Selain itu, platform GitHub berfungsi sebagai media penyimpanan berbasis cloud yang memudahkan akses berkas praktikum serta mendukung kolaborasi pengembangan proyek web di masa mendatang.